

An Implementation of the Conjugate Gradient Algorithm on FPGAs

David DuBois¹, Andrew DuBois¹, Thomas Boorman¹, Carolyn Connor¹, Steve Poole²

Los Alamos National Laboratory¹, Oak Ridge National Laboratory²

¹[dhd, ajd, tmb, connor]@lanl.gov, ²spoole@ornl.gov

Abstract

The conjugate gradient is a prominent iterative method for solving systems of sparse linear equations. Large-scale scientific applications often utilize a conjugate gradient solver at their computational core. Since a single iteration of a conjugate gradient solver requires a sparse matrix-vector multiply operation it is imperative that this operation be computed efficiently. In this paper we present a field programmable gate array (FPGA) based implementation of a double precision, non-preconditioned, conjugate gradient solver for finite-element or finite-difference methods. We show that our FPGA implementation can outperform current generation processors while running at a ~30X slower clock rate. Our work utilizes the SRC Computers, Inc. MAPStation hardware platform along with the “Carte” software programming environment.

Los Alamos National Laboratory, an affirmative action/equal opportunity employer, is operated by the Los Alamos National Security, LLC for the National Nuclear Security Administration of the U.S. Department of Energy under contract DE-AC52-06NA25396. By acceptance of this article, the publisher recognizes that the U.S. Government retains a nonexclusive, royalty-free license to publish or reproduce the published form of this contribution, or to allow others to do so, for U.S. Government purposes. Los Alamos National Laboratory requests that the publisher identify this article as work performed under the auspices of the U.S. Department of Energy. Los Alamos National Laboratory strongly supports academic freedom and a researcher's right to publish; as an institution, however, the Laboratory does not endorse the viewpoint of a publication or guarantee its technical correctness. This paper is published under LA-UR-08-1646.

1 INTRODUCTION

The Conjugate Gradient Method (CG) is a member of a family of iterative solvers known as Krylov subspace methods used primarily on large sparse linear systems arising from the discretization of partial differential equations (PDEs). CG is effective for systems of the form:

$$A\vec{x} = \vec{b},$$

where A is a square N x N sparse matrix [2].

To compress the sparse matrix A, the ELLPACK-ITPACK format was chosen [4]. This format is efficient for the matrix-vector multiply operation and performs well on vector style architectures.

Each successive iteration of the CG involves one Sparse Matrix-Vector Multiplication (SMVM), three vector updates, and two inner products. The SMVM is the time dominant computational portion executed per iteration of the CG.

2 SRC SYSTEM

The SRC MAPStation and SNAP Interface

The SRC MAPStation [1][3] utilizes a dual Intel Xeon motherboard. Each Xeon processor runs at 2.8 GHz, and has

a 512 KB L2 cache. A single SNAP module is inserted into one of the DDR memory slots and is connected to a single MAP processor module.

3 THE CONJUGANT GRADIENT

The non-preconditioned CG implementation we present in this work is shown in the pseudo code of Figure 1.

```

 $\vec{r} \leftarrow \vec{b} - A\vec{x} \quad (1)$ 
 $\vec{d} \leftarrow \vec{r}$ 
 $\delta_{new} \leftarrow \vec{r}^T \vec{r} \quad (2)$ 
 $\delta_0 \leftarrow \delta_{new}$ 
 $i \leftarrow 0$ 
While  $i < i_{max}$  and  $\delta_{new} > \epsilon^2 \delta_0$  do
   $\vec{q} \leftarrow A\vec{d} \quad (3)$ 
   $\alpha_{accum} \leftarrow d^T \vec{q} \quad (4)$ 
   $\alpha \leftarrow \frac{\delta_{new}}{\alpha_{accum}}$ 
   $\vec{x} \leftarrow \vec{x} + \alpha \vec{d} \quad (5)$ 
   $\vec{r} \leftarrow \vec{r} - \alpha \vec{q} \quad (6)$ 
   $\delta_{old} \leftarrow \delta_{new}$ 
   $\beta \leftarrow \frac{\delta_{new}}{\delta_{old}}$ 
   $\delta_{new} \leftarrow \vec{r}^T \vec{r} \quad (7)$ 
   $\vec{d} \leftarrow \vec{r} + \beta \vec{d} \quad (8)$ 
   $i \leftarrow i + 1$ 
end while

```

• Figure 1 CG Pseudo Code

By evaluating the inner loop operations (3) – (8) of the pseudo code, assuming that they are fully pipelined vector operations, we can estimate the run-time of a single iteration of the non-optimized CG algorithm. Using the approximate number of clock cycles and floating point operations (FLOPS) we can calculate the estimated best-case sustained performance of this code would be ~191 MFLOPs (clock cycle of 10ns).

4 IMPLEMENTATION

In the CG pseudo code shown in Figure 1, all operations shown are implemented on the MAP with the exception of the initial computation of the residual vector (1).

We optimized data placement in the MAP On Board Memory (OBM) to remove conflicts and utilized striping to improve performance. The maximum rank of the A matrix which can be solved by this implementation is 116,394.

Both user logic devices of the MAP are used in this implementation. Figures 2 and 3 contain detailed pseudo code specifying the operations performed by the primary and secondary FPGA devices. In these figures operations

(1), (2), and (3) are all computed in parallel greatly reducing run time.

```

DMA stripe A  $\Rightarrow$  AL0, AL1
DMA ja  $\Rightarrow$  jaL
DMA x  $\Rightarrow$  xeven

STREAM DMA r  $\Rightarrow$   $\left\{ \begin{array}{l} \text{reven} \\ \text{deven0} \\ \text{deven1} \\ \delta_{\text{new}} \leftarrow r^T r \end{array} \right\}$  (1)

 $\delta_0 \leftarrow \delta_{\text{new}}$ 
 $i \leftarrow 0$ 
 $\text{max\_iter} \leftarrow \text{max\_iteration}$ 
 $\text{converge\_value} \leftarrow \epsilon^2 \delta_0$ 
 $\text{send\_to\_bridge}(\text{max\_iter},$ 
 $\text{converge\_value},$ 
 $\text{nrows})$ 
 $\text{recv\_from\_bridge}()$  // handshake
While  $i < \text{max\_iter}$  and  $\delta_{\text{new}} > \text{convergence\_value}$ 
 $\left\{ \begin{array}{l} q \leftarrow A d_{\text{even\_odd}} \\ \alpha_{\text{accum}} \leftarrow d_{\text{even\_odd}}^T q \end{array} \right\}$  (2)
 $\alpha \leftarrow \frac{\delta_{\text{new}}}{\alpha_{\text{accum}}}$ 
 $\text{send\_perms}(\text{ALL\_OBM})$ 
 $\text{send\_to\_bridge}(\alpha, \delta_{\text{new}})$ 
 $\text{recv\_from\_bridge}(\delta_{\text{new}})$ 
 $\text{send\_perms}(0)$ 
 $i \leftarrow i + 1$ 
end while
DMA x  $\leftarrow$  xeven or xodd
DMA r  $\leftarrow$  reven or rodd

```

• Figure 2 Primary FPGA

```

recv_from_bridge(max_iter, converge_value, nrows)
send_to_bridge() // send ACK
i  $\leftarrow$  0
 $\delta_{\text{new}} \leftarrow 1.0$  // initial value
While  $i < \text{max\_iter}$  and  $\delta_{\text{new}} > \text{converge\_value}$  do
   $\text{recv\_perms}()$  // memory access
   $\text{recv\_from\_bridge}(\alpha, \delta_{\text{new}})$  // memory access
   $\left\{ \begin{array}{l} x_{\text{odd\_even}} \leftarrow x_{\text{even\_odd}} + \alpha d_{\text{even\_odd}} \\ r_{\text{odd\_even}} \leftarrow r_{\text{even\_odd}} - \alpha q \\ r_{\text{accum}} \leftarrow r_{\text{even\_odd}}^T r_{\text{even\_odd}} \end{array} \right\}$  (3)
   $\delta_{\text{old}} \leftarrow \delta_{\text{new}}$ 
   $\delta_{\text{new}} \leftarrow r_{\text{accum}}$ 
   $\beta \leftarrow \frac{\delta_{\text{new}}}{\delta_{\text{old}}}$ 
   $d_{\text{odd\_even}} \leftarrow r_{\text{even\_odd}} + \beta d_{\text{even\_odd}}$  (4)
   $\text{send\_to\_bridge}(\delta_{\text{new}})$ 
   $\text{recv\_perms}()$ 
   $i \leftarrow i + 1$ 
end while

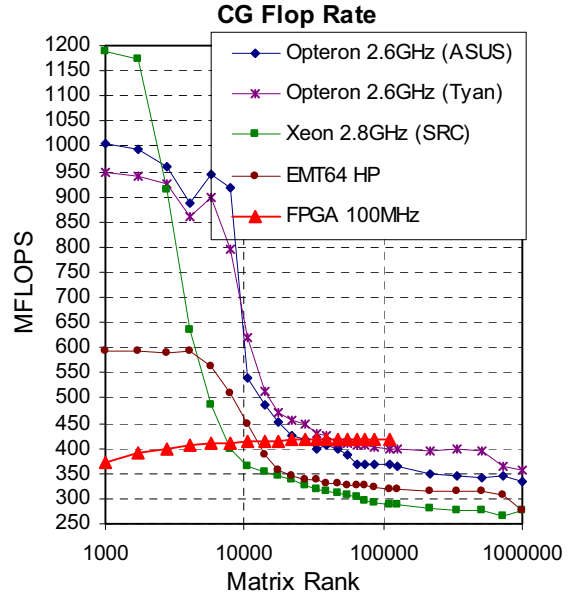
```

• Figure 3 Secondary FPGA

5 RESULTS

For comparative numbers, we generated a version of the non-preconditioned CG code to run on various general purpose machines. These results are shown in Figure 4.

For problem sizes where $N \approx 8,000$ and greater, we see that the MAP will attain a maximum sustained processing rate of 418 MFlops for the CG. All the CPU based systems perform reasonably well for small problems sizes since the requisite data structures can fully reside in cache. The FPGA based implementation outperforms all of the CPU based systems for problem sizes larger than $N = 46,656$ and offers deterministic performance.



• Figure 4 Results

6 CONCLUSION

We have presented a design that implements an efficient CG algorithm on the SRC MAPStation. We have demonstrated that an FPGA based system can outperform today's processors while running over 30 times slower (i.e. 100 MHz vs. 3.4 GHz). This is possible because an FPGA-based system can be designed to more optimally match the computational units to available memory bandwidth providing a more balanced system. It also provides parallelism not available in any standard microprocessor design and allows unique functionality such as vector replication in a single clock cycle.

7 REFERENCES

- [1] SRC Computers, Inc. Product Page, October, 1999-2004. <<http://www.srccomputers.com/Products.htm>>
- [2] Shewchuk, Jonathan, "An Introduction to the Conjugate Gradient Method Without the Agonizing Pain," *School of Computer Science, Carnegie Mellon University*, 1994.
- [3] SRC Computers, Inc. SRC C Programming Environment v2.1 Guide. SRC Computers, Inc. August 31, 2005.
- [4] R.T. Mills, E.F. D'Azevedo, and M.R. Fahey., "Progress Towards Optimizing the PETSc Numerical Toolkit on the Cray X1," Cray Users Group, May, 2005.